Análise e Desen. de Sistemas

TO WORK

PLATAFORMA WEB PARA CONEXÃO ENTRE CLIENTES E PRESTADORES DE SERVIÇOS

RYCHARD VINICIOS FONDELIS DA SILVA
GABRIEL LUIZ DE OLIVEIRA

Projeto Interdisciplinar
Mentor: Prof. Willian Aparecido Hübner do Nascimento
2026
Sinop-MT

CENTRO UNIVERSITÁRIO UNIFASIPE JET

SILVA, Rychard Vinicios Fondelis da; OLIVEIRA, Gabriel Luiz de. To Work: plataforma web para conexão entre clientes e prestadores de serviços. Sinop-MT, 2026. Projeto Interdisciplinar – Centro Universitário Fasipe – UNIFASIPE.

RESUMO

Este trabalho apresenta o desenvolvimento do To Work, uma plataforma web destinada a aproximar clientes e prestadores de serviços. O objetivo foi construir um fluxo de atendimento desde a descoberta do profissional até a avaliação de um serviço concluído, com uma interface consistente e uma estrutura adequada ao aprendizado. A metodologia reuniu análise do ciclo de atendimento, levantamento de requisitos, modelagem de dados, implementação incremental e testes automatizados. Foram utilizados React com Vite na interface, Node.js com Express na API e SQLite na persistência. A versão implementada oferece cadastro, autenticação, pesquisa por texto, cidade e categoria, edição do perfil profissional, solicitação de atendimento, orçamento, aceite, conclusão e avaliação. O catálogo padrão contém 152 categorias organizadas em 12 áreas, com busca por nomes de profissões e possibilidade de vincular várias áreas ao prestador. O painel permite escolher categorias existentes e controlar seus vínculos, sem apresentar um formulário de criação de categorias. A API verifica os participantes e as transições permitidas antes de alterar uma solicitação. A validação compreendeu oito testes de API, quatro testes de navegador, construção da interface e auditoria das dependências. Os cenários executados foram aprovados, incluindo consulta ao catálogo, proteção de pedidos de terceiros e recuperação de falhas de conexão. Conclui-se que a aplicação atende ao percurso acadêmico proposto com tecnologias acessíveis, mantendo publicação segura, pagamentos, agenda e comunicação entre participantes como possibilidades futuras.

Palavras-chave: autenticação; desenvolvimento web; orçamento; prestação de serviços; React.

ABSTRACT

This work presents To Work, a web platform that connects clients and service providers through an accessible academic architecture. The goal was to implement the workflow from professional discovery to the review of a completed service. The method included service workflow analysis, requirements identification, data modeling, incremental implementation and automated testing. The application uses React with Vite, Node.js with Express and SQLite. Features include registration, authentication, search, professional profiles, service requests, quotations, acceptance, completion and reviews. A standard catalog provides 152 categories organized into 12 groups, with profession search and multiple categories per provider. Providers select existing categories and manage their links; category creation is not available as a dashboard form. The server checks permissions and status transitions. Eight API tests and four browser tests passed, and the frontend build and dependency audits completed. The application supports the proposed academic workflow, while payments, scheduling and participant messaging remain future developments.

Keywords: authentication; quotation; React; service providers; web development.

HISTÓRICO DA REVISÃO

Data	Descrição	Responsabilidade
10/10/2025	Levantamento dos requisitos funcionais.	Autoria do projeto
12/10/2025	Definição dos requisitos não funcionais.	Autoria do projeto
15/10/2025	Elaboração de fluxogramas e protótipos iniciais.	Autoria do projeto
08/06/2026	Documentação da estrutura React, Express e SQLite.	Autoria do projeto
30/10/2026	Consolidação do fluxo de serviços, catálogo, interface, testes e documentação.	Autoria do projeto

Fonte: elaboração a partir do código e das verificações do projeto (2026).

A versão documentada reúne o fluxo de atendimento, a interface responsiva e o catálogo padrão de 152 categorias em 12 áreas. O painel permite vincular categorias existentes ao perfil profissional. As evidências de validação correspondem aos cenários executados em 30/10/2026.

SUMÁRIO

1 INTRODUÇÃO 7

1.1 Problemática 7

1.2 Justificativa 7

1.3 Objetivos 8

1.3.1 Geral 8

1.3.2 Específicos 8

1.4 Metodologia 9

2 REVISÃO DE LITERATURA 9

2.1 Tópicos envolvidos 9

2.1.1 Interface com componentes e ferramentas de construção 9

2.1.2 API e validação no servidor 9

2.1.3 Banco relacional e transações 10

2.1.4 Hash de senhas, JWT e autorização 10

2.1.5 Qualidade e simplicidade 11

2.2 Comparativo de tecnologias 11

2.2.1 React e páginas estáticas 11

2.2.2 SQLite e banco cliente-servidor 11

3 MATERIAIS E MÉTODOS 12

3.1 Coleta e análise de requisitos 12

3.1.1 Catálogo de profissões e serviços 13

3.2 Requisitos 14

3.2.1 Requisitos funcionais 14

3.2.2 Requisitos não funcionais 15

3.3 Metodologia de desenvolvimento 16
3.4 Casos de uso 16
3.5 Diagrama de Entidade e Relacionamento 18
3.6 Arquitetura de software 19
3.7 Fluxogramas e situações do serviço 20
3.8 Acompanhamento das atividades 23
4 RESULTADOS E DISCUSSÕES 23
4.1 Telas da versão implementada 23
4.2 Testes de software 32
4.3 Riscos identificados 33
4.4 Discussão dos resultados 34
5 PERSPECTIVAS FUTURAS 35
6 CONSIDERAÇÕES FINAIS 36
REFERÊNCIAS BIBLIOGRÁFICAS 37
APÊNDICE A – EXECUÇÃO E DEMONSTRAÇÃO 38
A.1 Preparação local 38
A.2 Roteiro de demonstração 38
A.3 Verificação e estudo 39
APÊNDICE B – CATÁLOGO PADRÃO DE SERVIÇOS 40

1 INTRODUÇÃO

A contratação de serviços locais envolve encontrar um profissional, compreender sua oferta, negociar um valor e acompanhar o atendimento. Quando essas informações ficam distribuídas em indicações e conversas informais, o cliente tem dificuldade para comparar alternativas e manter um registro do que foi combinado. O To Work organiza esse processo em uma plataforma web com perfis, categorias e solicitações de serviço.

O projeto utiliza uma aplicação de interface composta por componentes e uma API responsável pelas regras de negócio. Essa separação permite estudar, no mesmo trabalho, desenvolvimento de interfaces, comunicação HTTP e persistência relacional. Componentes e estado são conceitos centrais do modelo de construção de interfaces apresentado pela documentação do React (REACT, 2026).

A aplicação reúne cadastro e autenticação, busca de profissionais, perfil completo, orçamentos e avaliações em um ciclo de atendimento. Os recursos foram organizados de forma incremental, com tecnologias acessíveis ao desenvolvimento acadêmico. O sistema registra a negociação e o andamento, sem processar pagamentos ou verificar presencialmente a realização do serviço.

1.1 Problemática

Como oferecer a clientes e profissionais um percurso organizado entre a pesquisa e o encerramento de um serviço, com uma estrutura adequada ao desenvolvimento acadêmico? A solução precisa integrar a descoberta do profissional, a negociação e o acompanhamento do pedido, mantendo os registros e as permissões de cada participante.

1.2 Justificativa

A proposta reúne em um único ambiente a divulgação profissional, a descrição da necessidade do cliente, o orçamento e a avaliação. Cada solicitação registra sua situação atual e o valor informado; a versão não mantém uma trilha completa de alterações anteriores. Essa organização permite demonstrar persistência e controle de acesso sem depender de integrações externas no fluxo principal.

No contexto acadêmico, o desenvolvimento integra modelagem relacional, validação de dados, programação com componentes, autenticação e testes. React, Express e SQLite permitem organizar essas responsabilidades com uma estrutura de fácil compreensão e manutenção. A simplicidade foi adotada como critério de desenho, em consonância com Google Engineering Practices (GOOGLE, 2026).

1.3 Objetivos

1.3.1 Geral

Desenvolver e documentar uma plataforma web para conectar clientes e prestadores, permitindo pesquisar profissionais e acompanhar uma solicitação desde seu cadastro até a conclusão e a avaliação.

1.3.2 Específicos

• Oferecer uma identidade visual consistente entre as telas, mantendo uma arquitetura simples.

• Implementar pesquisa por texto, cidade, categoria e termos de profissões.

• Organizar 152 categorias padrão em 12 áreas, com seleção de múltiplas atividades por prestador.

• Permitir ao prestador editar informações profissionais e ativar ou desativar vínculos de categorias existentes.

• Implementar solicitações, orçamentos e transições de situação com autorização no servidor.

• Vincular avaliações a serviços concluídos, impedindo duplicação.

• Verificar os fluxos por testes de API e navegador e registrar evidências reproduzíveis.

• Documentar o comportamento atual e as possibilidades futuras segundo o modelo institucional.

1.4 Metodologia

Foi adotada uma abordagem aplicada e incremental. O desenvolvimento compreendeu a definição do ciclo de atendimento, o levantamento de requisitos, a modelagem dos dados e a implementação das regras na API e na interface. A validação utilizou testes automatizados com bancos temporários, capturas de telas e conferência da migração em uma cópia do banco local.

Os requisitos foram definidos a partir dos papéis de cliente e prestador e das operações necessárias para pesquisar, solicitar, negociar, concluir e avaliar um serviço. Não foram realizadas entrevistas, pesquisa de satisfação ou observação de usuários externos. As conclusões de funcionamento se limitam aos cenários documentados.

2 REVISÃO DE LITERATURA

2.1 Tópicos envolvidos

2.1.1 Interface com componentes e ferramentas de construção

Uma interface em React é formada por componentes que combinam apresentação e lógica de interação. No To Work, páginas organizam a navegação e componentes representam áreas reutilizáveis, como a barra superior, o perfil editável e as solicitações. O estado acompanha os dados dos formulários e das respostas da API; o contexto compartilha a sessão entre telas (REACT, 2026). O Vite executa o ambiente de desenvolvimento e produz os arquivos estáticos da interface no comando de build (VITE, 2026).

2.1.2 API e validação no servidor

A API recebe requisições HTTP e responde com dados JSON. O método GET consulta, POST cria, PUT substitui as informações editáveis do perfil e PATCH altera a situação do pedido ou vínculo de categoria. O Express organiza essas rotas e os middlewares. A documentação de segurança do Express recomenda atenção aos dados recebidos, à configuração e às dependências, orientando a adoção de validação e tratamento controlado de erros (EXPRESS, 2026).

2.1.3 Banco relacional e transações

No modelo relacional, tabelas armazenam entidades e chaves estrangeiras conectam registros. A relação entre profissionais e categorias é muitos para muitos e exige uma tabela de associação. O SQLite mantém a persistência em um arquivo, reduzindo a configuração necessária à demonstração. Sua documentação distingue essa finalidade da administração centralizada e da concorrência de servidores de banco de dados (SQLITE, 2026a).

Uma transação agrupa operações que precisam ocorrer juntas. BEGIN inicia a unidade, COMMIT confirma e ROLLBACK desfaz as alterações diante de falha. O To Work utiliza transações para cadastrar usuário e perfil e para complementar o catálogo durante a inicialização. Cada unidade utiliza uma conexão própria, evitando intercalar comandos de requisições diferentes (SQLITE, 2026b).

2.1.4 Hash de senhas, JWT e autorização

O bcrypt produz um hash usado na comparação de senhas. Hash não é criptografia reversível: a aplicação não oferece recuperação da senha original. A implementação utilizada limita a entrada a 72 bytes, conforme a característica do algoritmo documentada pelo projeto bcrypt.js (BCRYPT.JS, 2026). O custo configurado é 10, preservando uma escolha simples para a demonstração.

O JWT é emitido após cadastro ou login e validado nas rotas protegidas. A biblioteca jsonwebtoken permite definir expiração e algoritmos aceitos; nesta aplicação, o token expira em sete dias e usa HS256 (AUTH0, 2026). Autenticar identifica o usuário; autorizar determina se ele pode executar uma ação. Por isso, o servidor também confere os participantes e o estado do pedido.

2.1.5 Qualidade e simplicidade

Google Engineering Practices orienta a avaliação de desenho, funcionalidade, complexidade, testes, nomes, comentários e documentação. A recomendação de evitar complexidade desnecessária foi aplicada mantendo poucas camadas, regras explícitas e testes dos comportamentos relevantes (GOOGLE, 2026). As orientações são usadas como referência de revisão; o trabalho não apresenta certificação de qualidade nem comprovação de conformidade com uma norma específica.

2.2 Comparativo de tecnologias

2.2.1 React e páginas estáticas

Páginas estáticas atendem bem à apresentação de conteúdo fixo. O fluxo proposto exige dados de sessão, formulários e listas que mudam conforme a API. React já estava na base e permite organizar essas interações em componentes, evitando uma reescrita. A comparação é qualitativa; não foi executado benchmark entre frameworks ou entre React e HTML estático (REACT, 2026).

2.2.2 SQLite e banco cliente-servidor

SQLite simplifica a instalação e a distribuição local, pois dispensa um serviço separado. Bancos cliente-servidor podem ser mais adequados quando há necessidade de administração centralizada e muitas escritas concorrentes. A escolha depende da carga e da operação, e não de uma afirmação genérica de maior velocidade. A documentação do SQLite fundamenta essa distinção (SQLITE, 2026a).

Para produzir evidência da implementação escolhida, foi executada uma medição HTTP local com 100 prestadores fictícios, cinco requisições de aquecimento e cinquenta amostras sequenciais por rota. O ambiente utilizou Node.js v24.16.0, SQLite 3.52.0 e Windows. O tempo medido inclui a requisição e a leitura do JSON, sem concorrência.

Quadro 1 – Medição local da API

Rota	Mediana (ms)	P95 (ms)	Máximo (ms)
/health	2.12	6.58	15.47
/professionals	6.01	20.29	23.11
/professionals?q=eletrica&city=sinop	4.85	9.82	12.32

Fonte: elaboração a partir do código e das verificações do projeto (2026).

Esses valores descrevem apenas a execução local registrada em docs/evidencias/benchmark.json. P95 indica o valor abaixo do qual ficaram aproximadamente 95% das amostras. A medição não constitui teste de carga, garantia de desempenho em produção ou comparação experimental com outro banco. O script pode ser repetido com npm run benchmark --prefix backend.

3 MATERIAIS E MÉTODOS

3.1 Coleta e análise de requisitos

A análise de requisitos considerou os participantes, as informações do perfil profissional e as etapas da contratação. Foram definidas as ações permitidas ao visitante, ao cliente e ao prestador, os dados necessários a cada operação e as condições para alterar a situação de uma solicitação. A organização do documento segue o modelo acadêmico institucional.

O escopo inclui cadastro, autenticação, gestão de categorias, pesquisa e acompanhamento de serviços. O catálogo reúne 152 categorias padrão em 12 grupos, com pesquisa por profissões e vínculo de várias áreas ao perfil. A página inicial mostra seis destaques e oferece a lista completa na busca. Pagamento, chat, agenda, fotografias e recuperação por email permanecem como evoluções sem integração nesta versão.

3.1.1 Catálogo de profissões e serviços

O catálogo padrão contém 152 categorias em 12 grupos. Sua fonte é o arquivo serviceCategories.json. Na inicialização, a API insere apenas categorias ausentes, sem substituir descrições ou vínculos anteriores. As informações de grupo e os termos auxiliares são acrescentados à resposta da API a partir do JSON; não existe uma tabela de grupos no banco.

Quadro 2 – Composição do catálogo padrão

Área de atuação	Categorias padrão
Construção e reformas	24
Limpeza e organização	12
Jardim, campo e áreas externas	9
Instalação e manutenção	15
Tecnologia e informática	12
Transporte e serviços automotivos	12
Beleza e bem-estar	11
Eventos, alimentação e turismo	17
Aulas e educação	10
Cuidados, família e animais	9
Moda, costura e artesanato	8
Comunicação, design e negócios	13
TOTAL	152

Fonte: elaboração a partir do código e das verificações do projeto (2026).

No painel, a área “Com o que você trabalha?” permite pesquisar uma profissão, escolher uma categoria existente e vinculá-la ao perfil. A pesquisa desconsidera acentos e utiliza nome, descrição, grupo e termos auxiliares. Assim, jardineiro encontra Jardinagem e eletricista encontra Elétrica. Após cada vínculo, o prestador pode selecionar outra área; categorias já vinculadas deixam de aparecer entre as disponíveis.

Os seis cartões da página inicial são destaques de navegação. A lista completa permanece disponível no filtro de categoria. Categorias anteriores que não integram o arquivo padrão são preservadas e recebem o grupo Outros serviços. Por isso, uma base existente pode conter mais de 152 registros. O formulário “Cadastrar nova categoria” foi retirado do painel; a versão mantém uma rota de criação na API por compatibilidade, sem disponibilizar essa operação na interface atual.

3.2 Requisitos

3.2.1 Requisitos funcionais

Quadro 3 – Requisitos funcionais

REQ	Descrição implementada
RF01	Cadastrar cliente ou prestador, validando nome, email, senha e tipo de conta.
RF02	Impedir duplicação de email e guardar somente o hash da senha.
RF03	Criar o perfil profissional junto ao cadastro de prestador.
RF04	Autenticar por email e senha, emitir JWT e consultar a sessão ativa.
RF05	Proteger o painel e permitir sair da sessão do navegador.
RF06	Listar categorias e profissionais ativos publicamente.
RF07	Buscar profissionais por texto, termos de profissões, cidade e categoria ativa, sem distinção de acentos.
RF08	Exibir perfil público com localização, preço inicial, categorias e avaliações.
RF09	Permitir ao prestador editar telefone, cidade, UF, descrição e preço inicial.
RF10	Vincular categorias existentes ao próprio perfil e ativar ou desativar cada vínculo.
RF11	Permitir ao cliente solicitar serviço de uma categoria ativa do prestador.
RF12	Listar somente pedidos em que o usuário é cliente ou prestador.
RF13	Permitir ao prestador enviar orçamento positivo ou recusar pedido pendente.
RF14	Permitir ao cliente aceitar, recusar ou cancelar um pedido orçado e cancelar um pedido pendente.
RF15	Exibir telefone profissional ao cliente após o aceite e após a conclusão.
RF16	Permitir ao prestador concluir serviço aceito.

REQ	Descrição implementada
RF17	Permitir uma avaliação de 1 a 5 somente pelo cliente do serviço concluído.
RF18	Calcular avaliação pública a partir das avaliações vinculadas aos pedidos.
RF19	Filtrar solicitações por situação e atualizar a lista por ação do usuário.
RF20	Disponibilizar verificação de funcionamento da API com consulta ao banco.
RF21	Disponibilizar 152 categorias padrão em 12 grupos e complementar o banco sem duplicar ou apagar registros anteriores.
RF22	Agrupar a lista completa de categorias por área na busca pública e na seleção do prestador.
RF23	Pesquisar profissões no painel por nome, descrição, grupo e termos auxiliares; permitir múltiplas áreas por perfil.
RF24	Exibir seis categorias em destaque na página inicial e oferecer acesso à lista completa.
RF25	Apresentar no painel a seleção de categorias existentes, sem formulário para criar categorias.

Fonte: elaboração a partir do código e das verificações do projeto (2026).

3.2.2 Requisitos não funcionais

Quadro 4 – Requisitos não funcionais

REQ	Critério
RNF01	Manter cores, tipografia, cartões e estados consistentes entre as telas, com adaptação para celular e computador.
RNF02	Usar React/Vite, Node.js/Express e SQLite, mantendo uma estrutura acessível ao estudo.
RNF03	Validar dados na API, usar SQL parametrizado e respostas de erro controladas.
RNF04	Verificar token, conta ativa, papel, participantes e transições permitidas.
RNF05	Usar transações nas operações compostas e comparar o estado na atualização.
RNF06	Manter configurações em variáveis de ambiente e não versionar segredos.
RNF07	Limitar o JSON a 32 kB e o conjunto de tentativas de cadastro/login a 50 por IP em 15 minutos.
RNF08	Apresentar estados de espera, mensagens de erro, rótulos de formulário e alternativa de nova tentativa.
RNF09	Preservar registros existentes por migração aditiva e inicialização repetível do catálogo.
RNF10	Possuir testes reproduzíveis com bancos temporários e evidências documentadas.

REQ	Critério
RNF11	Usar ícones SVG locais e estilos compartilhados, sem exigir serviço externo para compor o visual.
RNF12	Conferir ausência de rolagem horizontal nas telas e larguras cobertas pelos testes de interface.

Fonte: elaboração a partir do código e das verificações do projeto (2026).

3.3 Metodologia de desenvolvimento

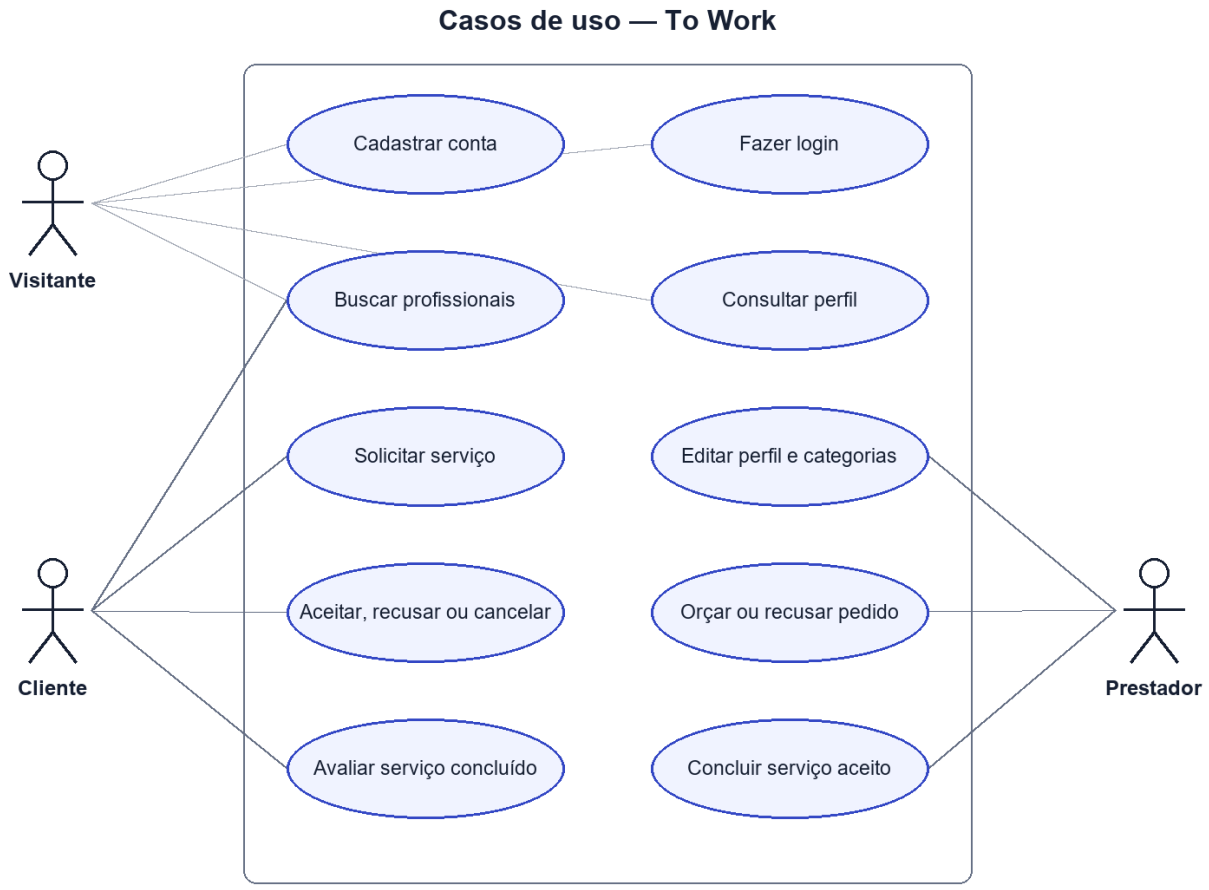
O desenvolvimento foi organizado em incrementos verificáveis: persistência de pedidos e avaliações; rotas e autorização; integração dos formulários; tratamento de falhas; testes e documentação. A base original foi copiada para uma pasta de backup antes das alterações. As dependências foram atualizadas dentro da estrutura existente; a biblioteca sqlite3 passou à versão 6, mantendo sua API de consultas utilizada no projeto.

A interpretação dos estados foi expressa em um conjunto curto de transições. A API valida essa sequência, sem depender somente dos botões visíveis. Operações concorrentes incluem o estado anterior na condição do UPDATE: se outra requisição já o alterou, a API devolve conflito. O valor contratado é guardado em centavos inteiros.

3.4 Casos de uso

O visitante consulta categorias e perfis e pode criar uma conta ou entrar. O cliente solicita e decide sobre orçamentos, acompanha seus pedidos e avalia serviços concluídos. O prestador mantém o perfil e as categorias, responde a solicitações e registra a conclusão. Ambos acessam apenas solicitações em que participam.

Figura 1 – Casos de uso do To Work



Fonte: Elaboração a partir da implementação (2026).

Quadro 5 – Descrição dos principais casos de uso

Caso	Condição e resultado
Solicitar serviço	Cliente autenticado escolhe categoria ativa, informa necessidade e cidade; a solicitação nasce PENDENTE.
Responder orçamento	Prestador do pedido pendente informa valor; situação muda para ORCADO.
Aceitar orçamento	Cliente do pedido orçado confirma; situação muda para ACEITO e telefone fica disponível.
Concluir serviço	Prestador do pedido aceito registra conclusão; situação muda para CONCLUIDO.

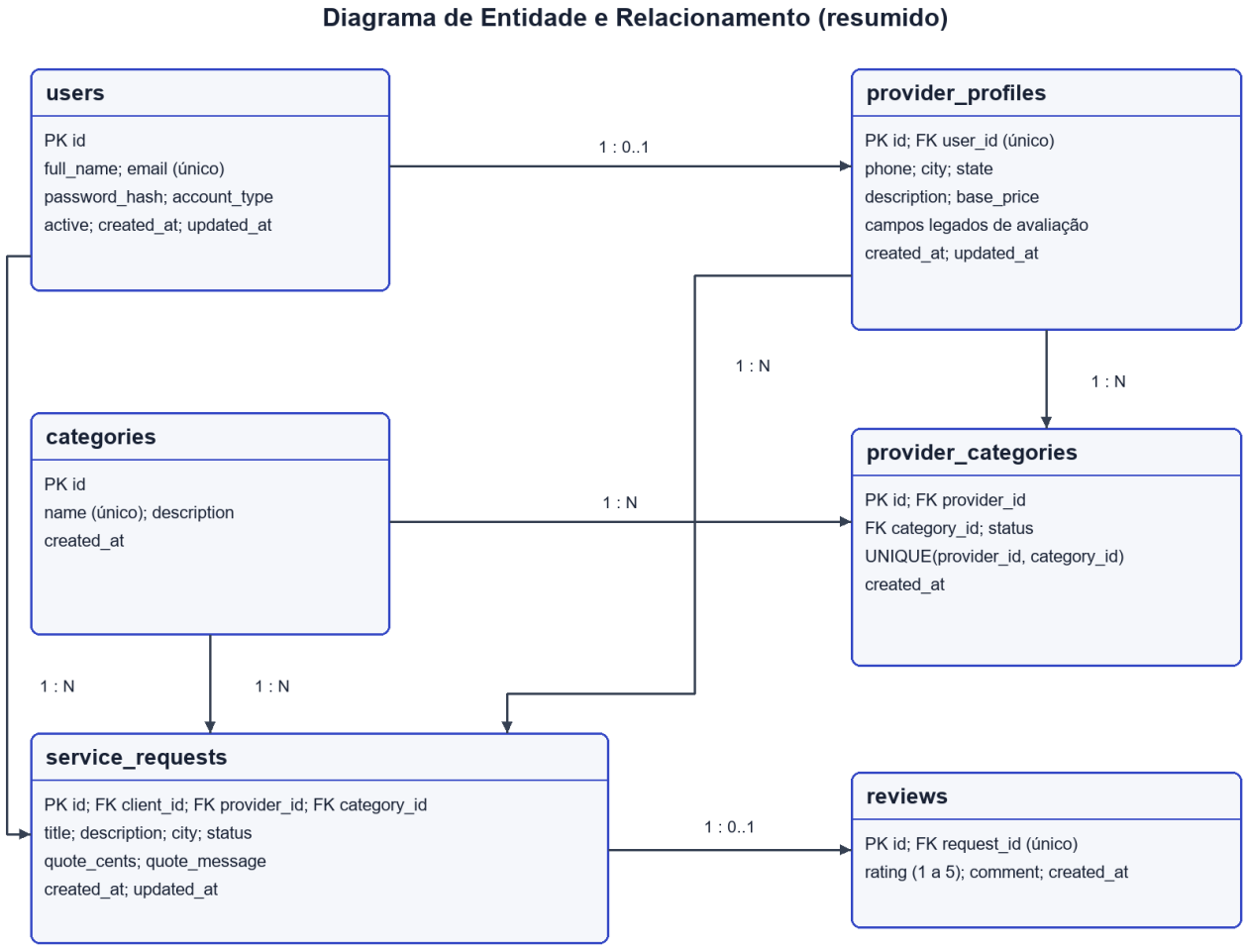
Caso	Condição e resultado
Avaliar serviço	Cliente do pedido concluído envia nota de 1 a 5; o vínculo único impede nova avaliação.

Fonte: elaboração a partir do código e das verificações do projeto (2026).

3.5 Diagrama de Entidade e Relacionamento

O banco contém seis tabelas. users armazena as contas. provider_profiles acrescenta os dados dos prestadores. categories organiza os serviços e provider_categories mantém a associação muitos para muitos. service_requests registra cliente, prestador, categoria, necessidade, situação e orçamento. reviews guarda no máximo uma avaliação por pedido.

Figura 2 – DER resumido do banco de dados



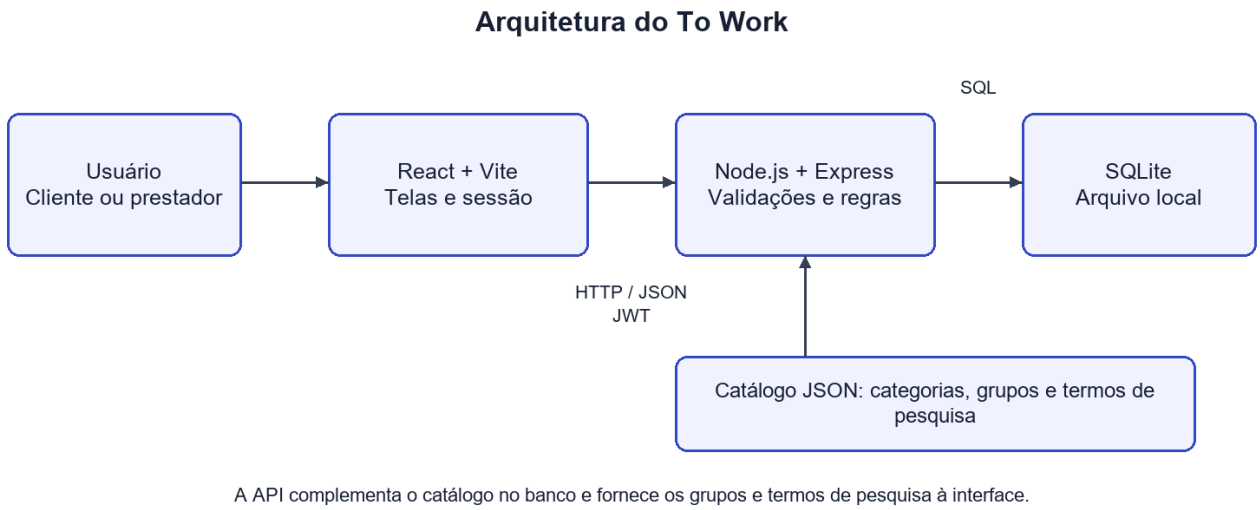
Fonte: Elaboração a partir da implementação (2026).

As chaves estrangeiras preservam os relacionamentos. O par prestador/categoria é único, assim como o email e o vínculo entre avaliação e pedido. As colunas legadas de nota e verificação permanecem no perfil para preservar a estrutura anterior, mas a nota pública atual é calculada por consultas às avaliações reais. O desenho é resumido; backend/schema.sql contém os campos e restrições completos.

3.6 Arquitetura de software

A interface é executada no navegador. O Axios centraliza o endereço da API, adiciona o token e limita o tempo de espera das chamadas. O Express distribui requisições para rotas de autenticação, vitrine, perfil e solicitações. O middleware de autenticação verifica a assinatura e a validade do JWT e consulta se a conta permanece ativa. As consultas ao SQLite utilizam parâmetros para os valores recebidos.

Figura 3 – Arquitetura da aplicação



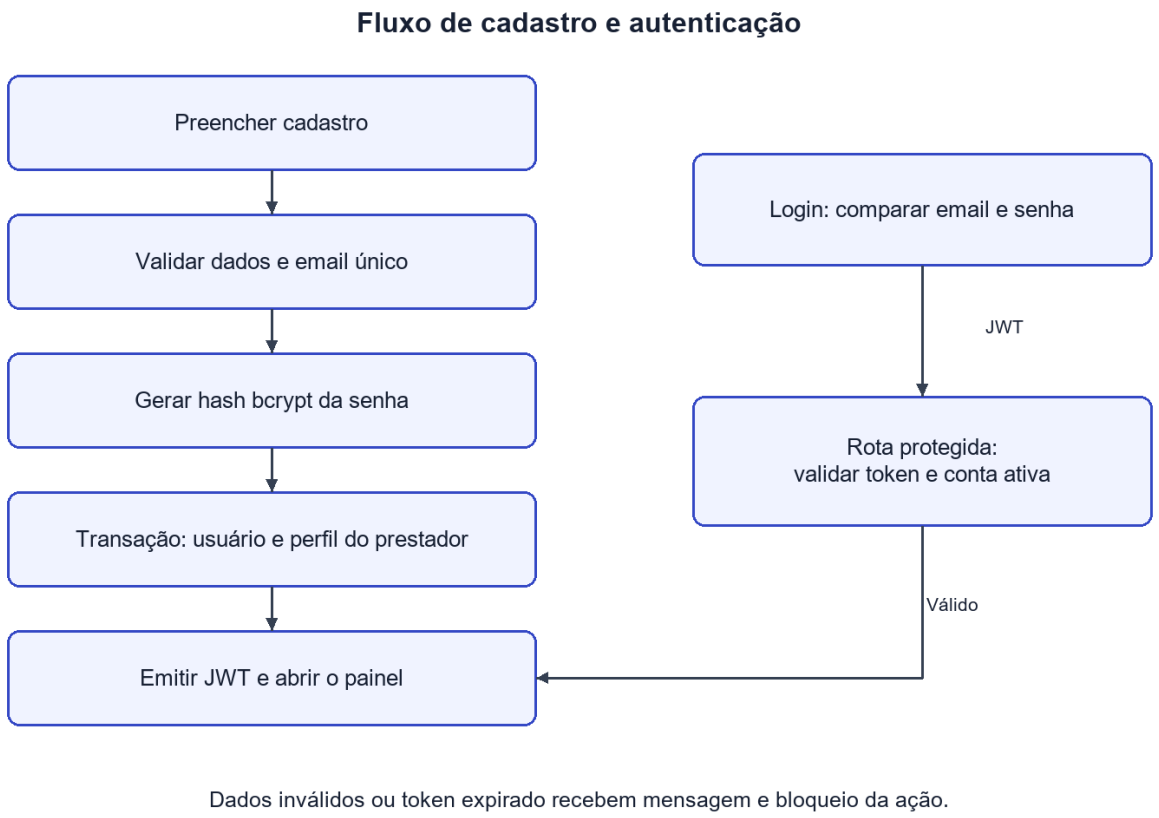
Fonte: Elaboração a partir da implementação (2026).

O módulo database.js concentra abertura, consultas, inicialização e transações. Em uma operação composta, a conexão exclusiva executa BEGIN IMMEDIATE, realiza as gravações e confirma com COMMIT; uma falha leva a ROLLBACK. Essa organização mantém a lógica de persistência identificável sem acrescentar um framework de banco de dados.

O catálogo JSON contém agrupamentos e descrições, enquanto SQLite mantém os identificadores e os vínculos utilizados nas solicitações. O componente CategoryOptions agrupa as opções recebidas da API em listas nativas do navegador. O compartilhamento de estilos e dos componentes de navegação, autenticação e ícones mantém o desenho consistente sem alterar essa separação de responsabilidades.

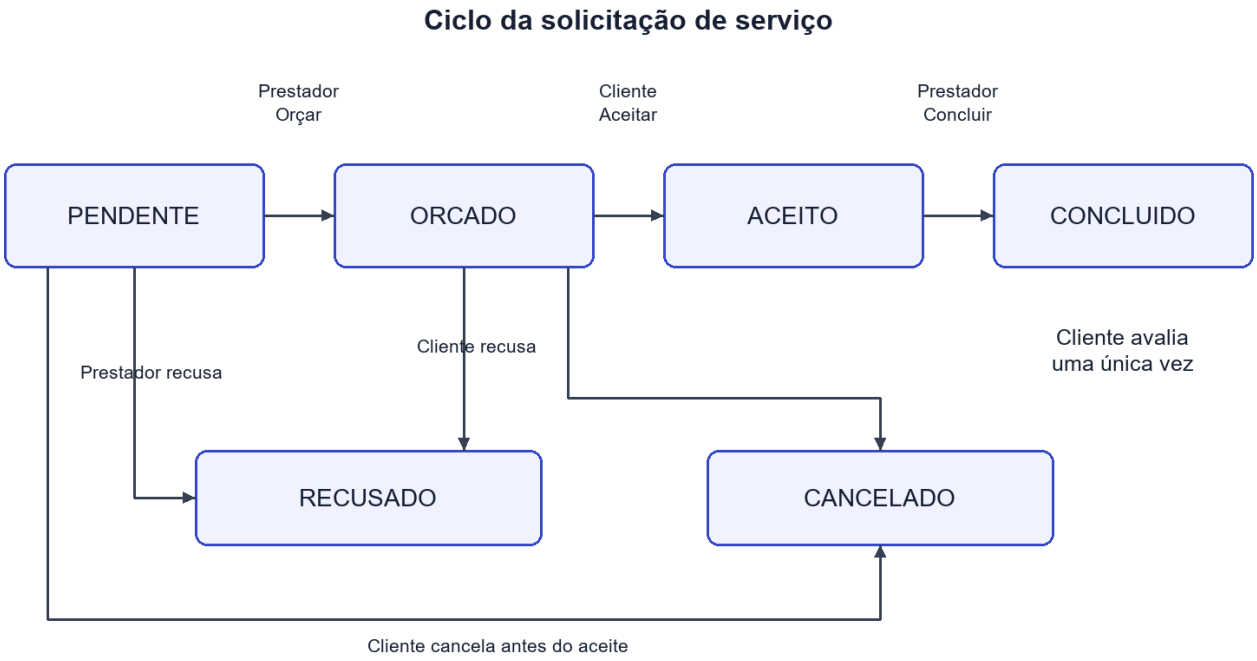
3.7 Fluxogramas e situações do serviço

Figura 4 – Fluxo de cadastro e autenticação



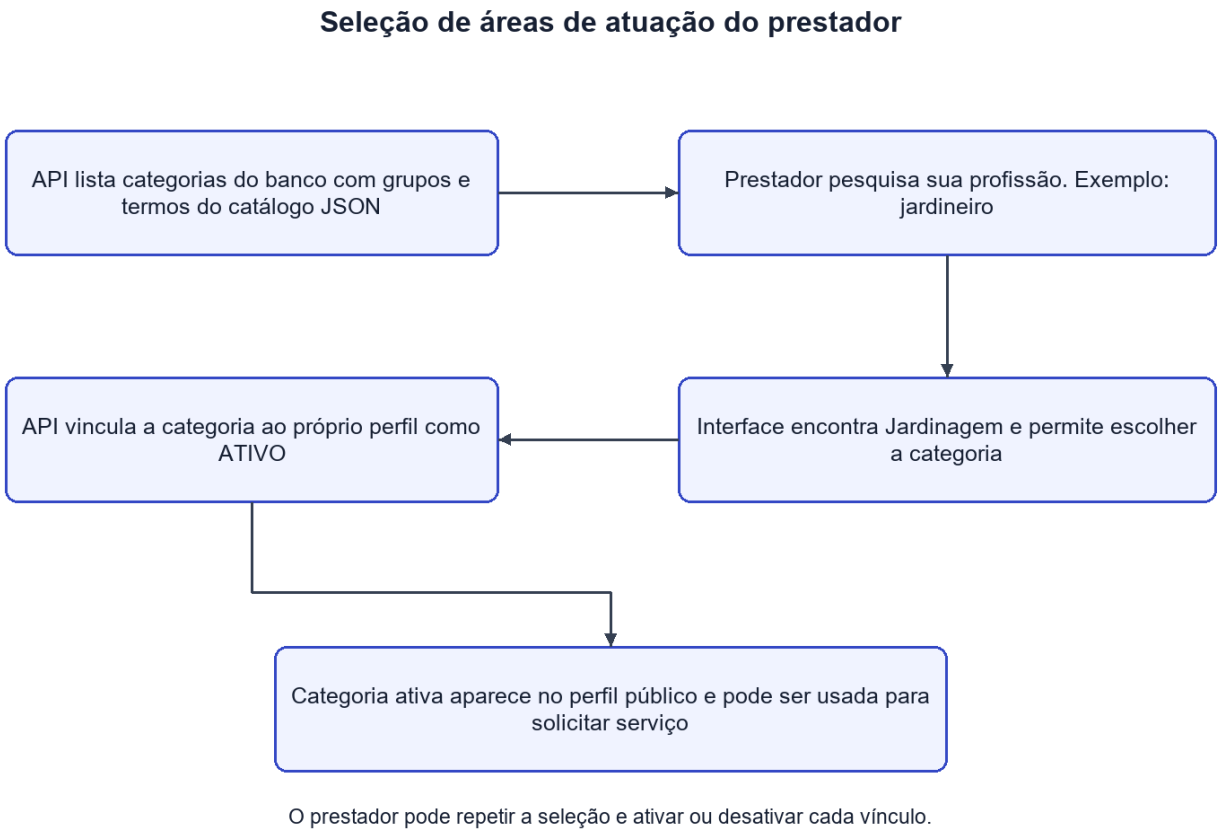
Fonte: Elaboração a partir da implementação (2026).

Figura 5 – Situações e transições de uma solicitação



Fonte: Elaboração a partir da implementação (2026).

Figura 6 – Pesquisa e vínculo de profissões no painel



Fonte: Elaboração a partir da implementação (2026).

Quadro 6 – Regras de transição

Situação atual	Responsável	Próximas situações
PENDENTE	Prestador	ORCADO ou RECUSADO
PENDENTE	Cliente	CANCELADO
ORCADO	Cliente	ACEITO, RECUSADO ou CANCELADO
ACEITO	Prestador	CONCLUIDO
CONCLUIDO / RECUSADO / CANCELADO	—	Sem novas transições; concluído permite avaliação única

Fonte: elaboração a partir do código e das verificações do projeto (2026).

3.8 Acompanhamento das atividades

Quadro 7 – Acompanhamento das atividades

Etapa	Data de consolidação	Situação
Análise de requisitos e organização do código	30/10/2026	Concluída
Implementação incremental dos fluxos	30/10/2026	Concluída
Testes de API e navegador	30/10/2026	Concluída
Medição local e conferência de migração	30/10/2026	Concluída
Documentação e roteiro de demonstração	30/10/2026	Concluída
Apresentação e avaliação acadêmica	A definir	Pendente
Publicação e integrações futuras	A definir	Fora do escopo atual

Fonte: elaboração a partir do código e das verificações do projeto (2026).

As datas indicam a consolidação das atividades da versão documentada. A apresentação e a avaliação acadêmica seguem o calendário da instituição.

4 RESULTADOS E DISCUSSÕES

4.1 Telas da versão implementada

As telas foram capturadas durante testes com usuários fictícios, em banco temporário. Após a implementação funcional, a interface recebeu uma revisão visual: azul-violeta, detalhes em amarelo, busca em faixa horizontal, ícones locais em SVG e cartões de profissionais com hierarquia de informações. Login, cadastro, perfil e painel compartilham cores e espaçamentos. O layout se adapta a celular e computador, mantendo os fluxos de pesquisa, orçamento e avaliação.

Figura 7 – Página inicial com busca funcional



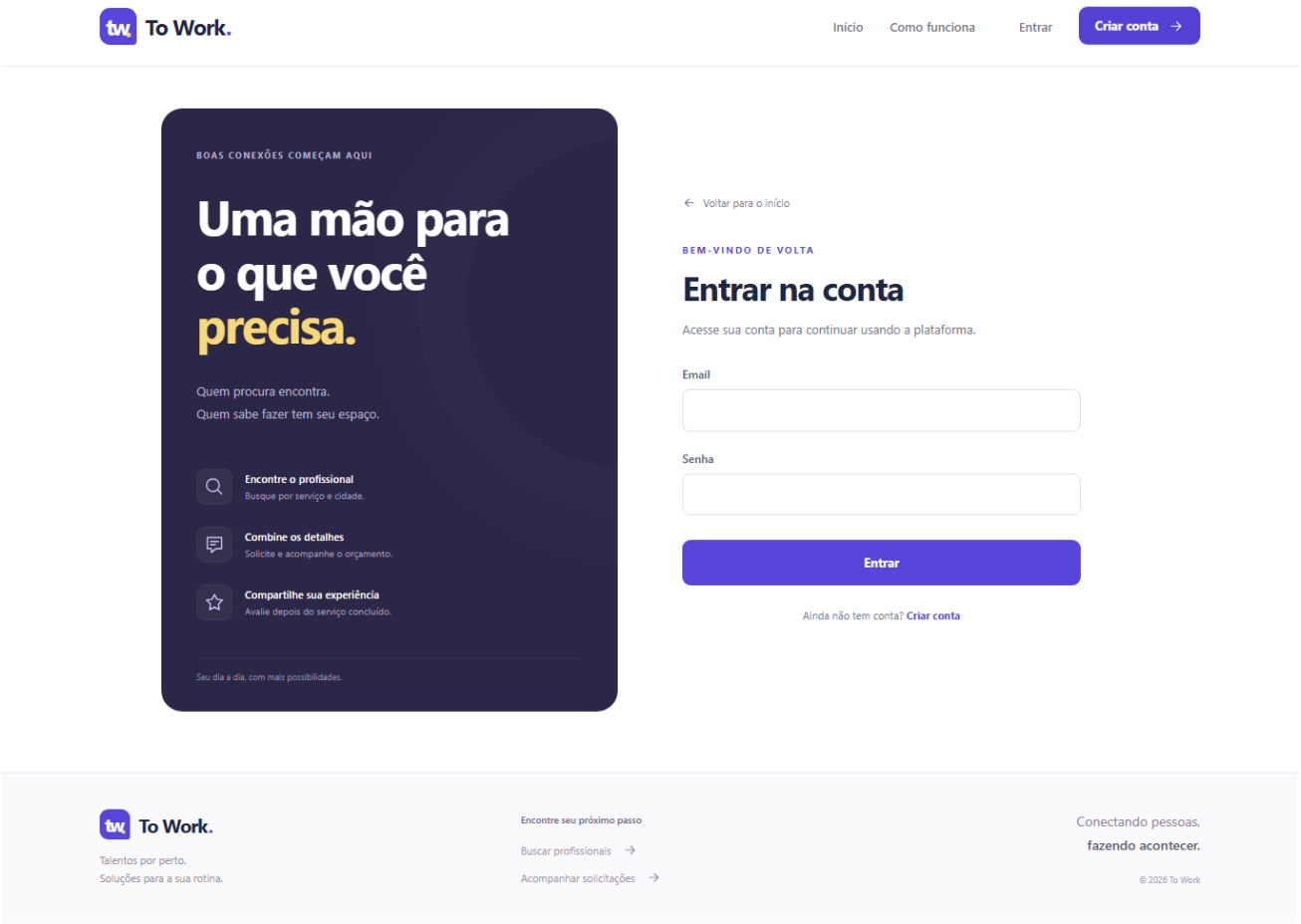
Fonte: Captura da aplicação executada em teste de navegador, com dados fictícios (2026).

Figura 8 – Cadastro de cliente ou prestador



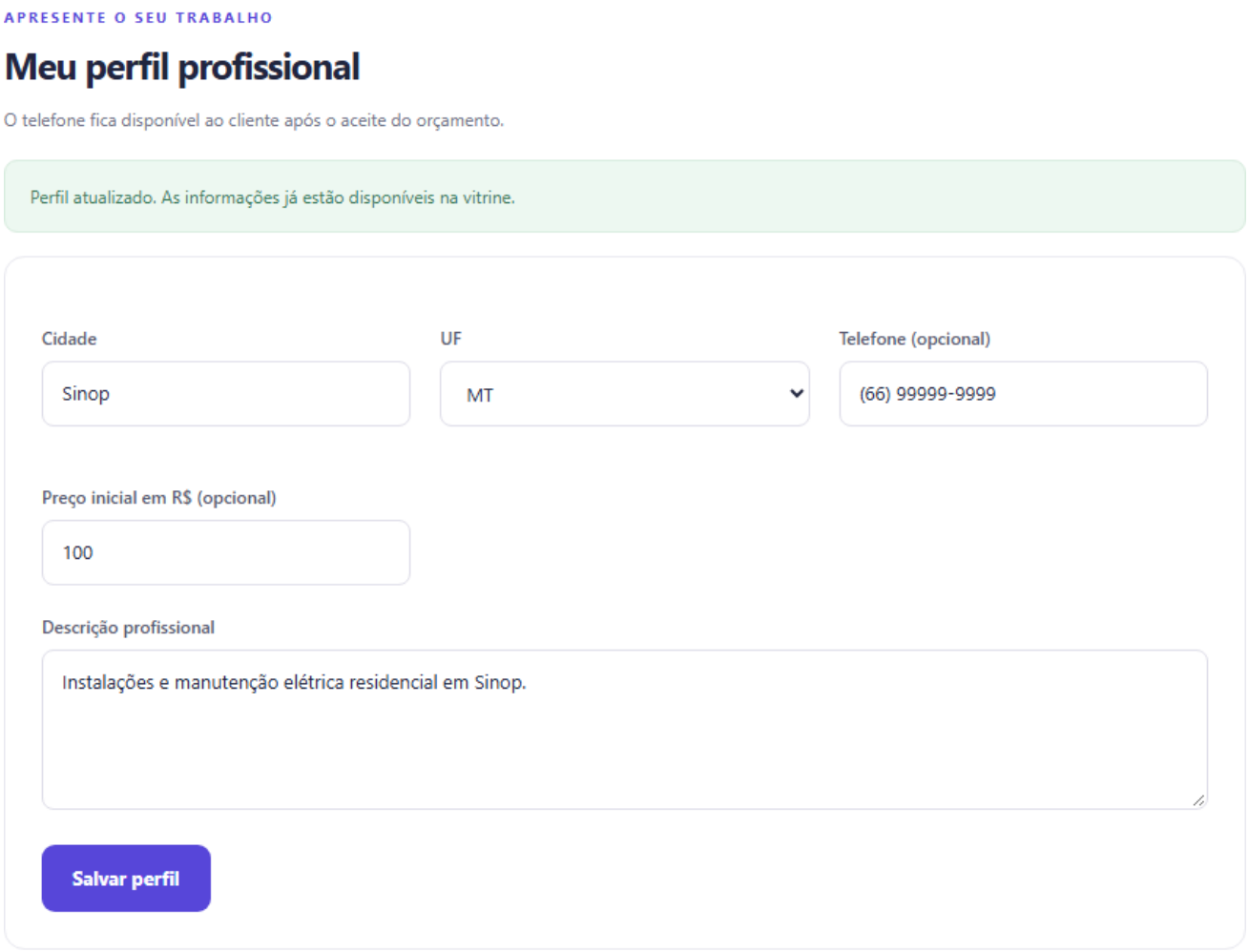
Fonte: Captura da aplicação executada em teste de navegador, com dados fictícios (2026).

Figura 9 – Autenticação de usuário



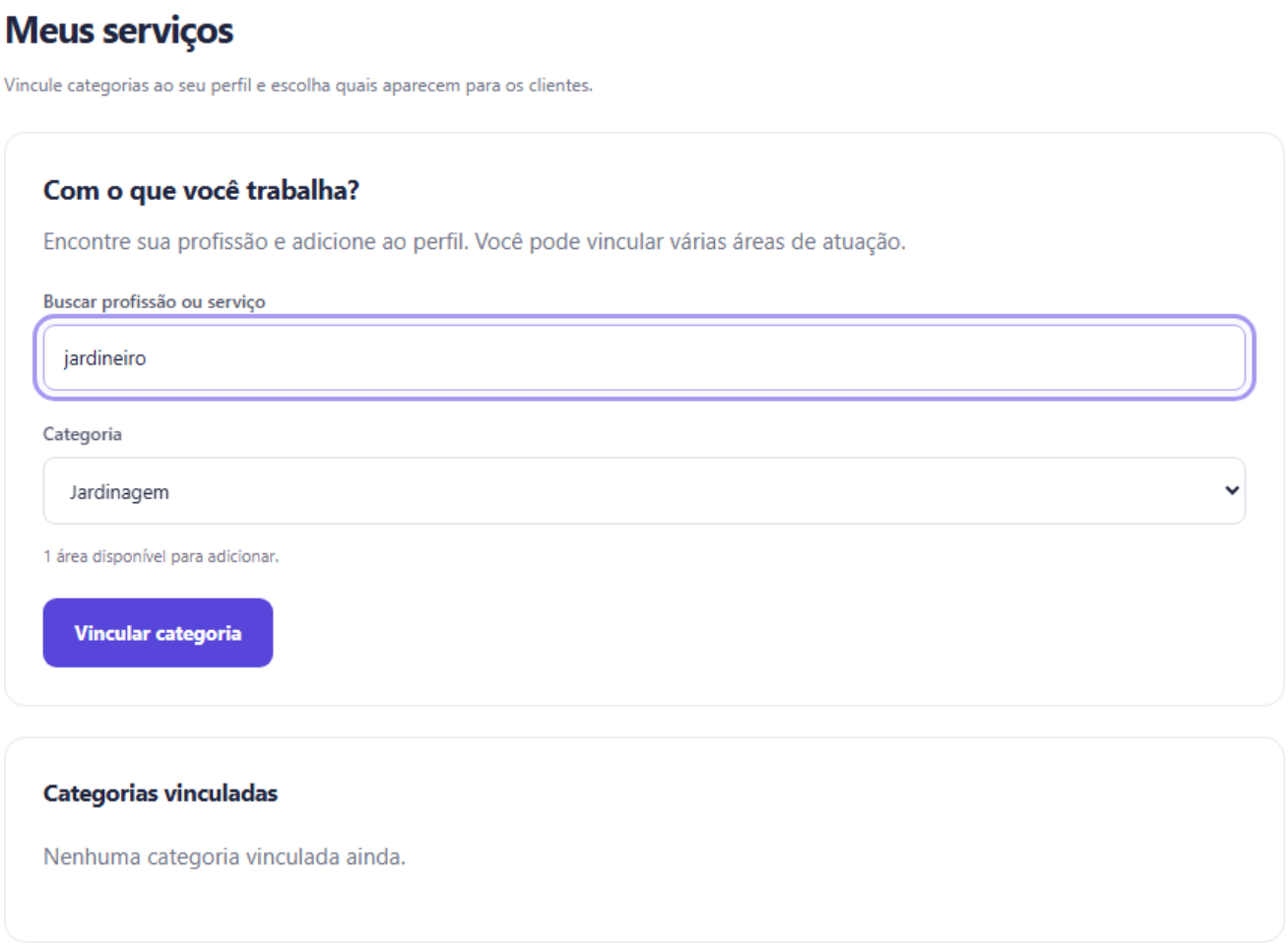
Fonte: Captura da aplicação executada em teste de navegador, com dados fictícios (2026).

Figura 10 – Painel do prestador e edição de perfil



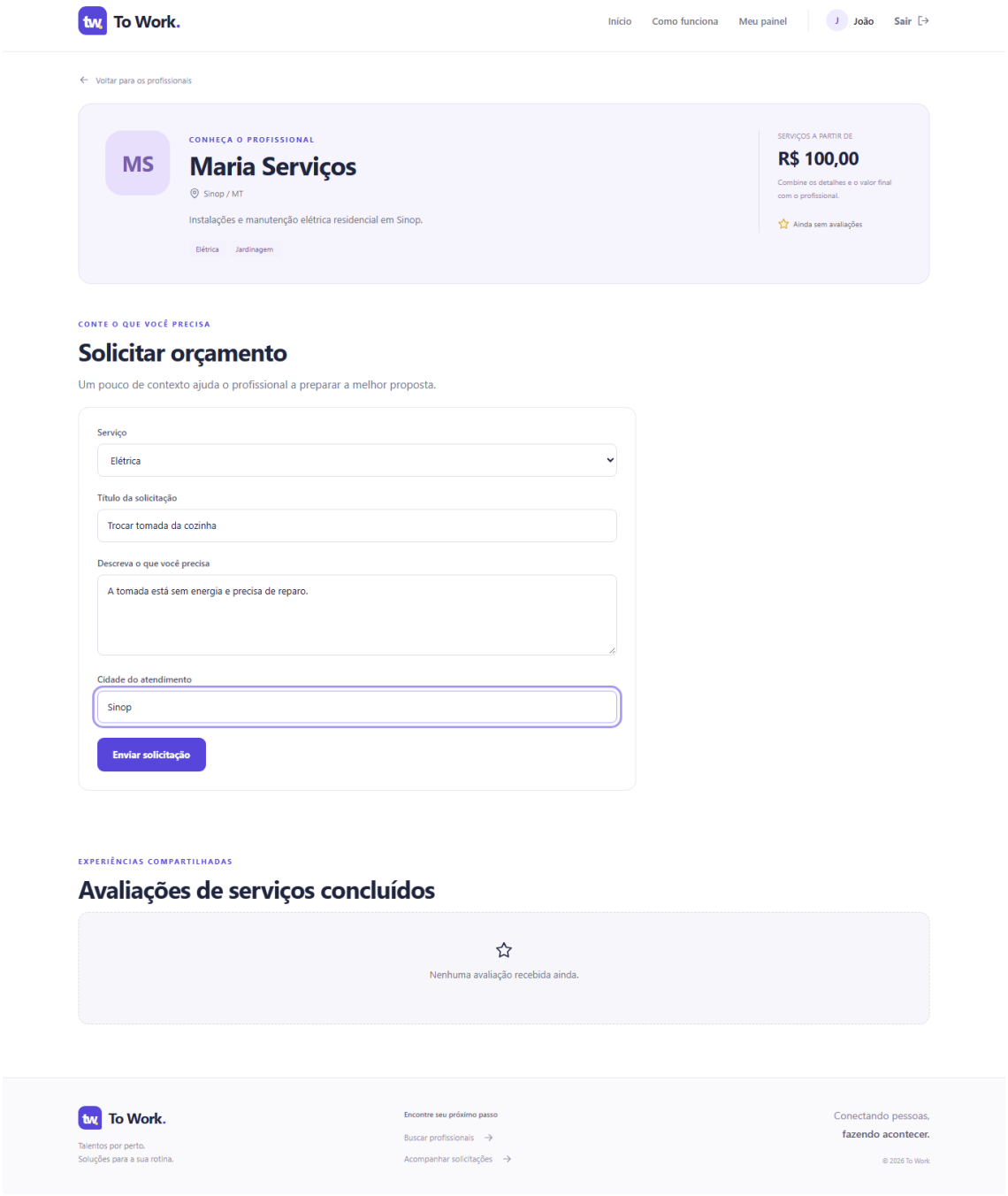
Fonte: Captura da aplicação executada em teste de navegador, com dados fictícios (2026).

Figura 11 – Seleção de profissão e categorias existentes



Fonte: Captura da aplicação executada em teste de navegador, com dados fictícios (2026).

Figura 12 – Perfil público e solicitação de serviço



Fonte: Captura da aplicação executada em teste de navegador, com dados fictícios (2026).

Figura 13 – Orçamento recebido pelo cliente



Fonte: Captura da aplicação executada em teste de navegador, com dados fictícios (2026).

O cliente pode aceitar ou recusar o orçamento recebido. Cancelar também é permitido enquanto o pedido não foi aceito. O valor informado no pedido é mantido em centavos inteiros no banco.

Figura 14 – Serviço concluído e avaliação registrada



Fonte: Captura da aplicação executada em teste de navegador, com dados fictícios (2026).

Figura 15 – Página inicial em largura de celular



Fonte: Captura da aplicação executada em teste de navegador, com dados fictícios (2026).

4.2 Testes de software

A validação foi executada com oito testes automatizados de API e quatro testes de navegador. Os testes da API utilizam o executor nativo do Node.js e exercitam requisições HTTP contra um servidor com banco temporário. Os testes de interface utilizam Playwright e portas separadas, simulando cliente e prestador em sessões independentes.

Quadro 8 – Testes automatizados executados

Teste	Cenário executado	Resultado
T01	Catálogo de 152 categorias, grupos e termos; inicialização repetida; migração e perfis antigos	Aprovado
T02	Cadastro, tipos, email, limite de senha, hash, login e expiração do JWT	Aprovado
T03	Perfil, busca sem acentos e por profissão, categoria inativa e privacidade pública	Aprovado
T04	Pedido, orçamento, aceite, conclusão e avaliação única	Aprovado
T05	Acesso a pedidos de terceiros, papéis e bloqueio de estados finais	Aprovado
T06	Decisões concorrentes: somente uma transição é aplicada	Aprovado
T07	Rollback após falha em operação composta	Aprovado
T08	Conta inativa e resposta controlada a JSON inválido	Aprovado
N01	Fluxo entre duas contas; pesquisa jardineiro; lista com 12 grupos; vínculo de várias áreas; reload e login	Aprovado
N02	Rota protegida, senha incorreta e token inválido	Aprovado
N03	Erro de rede e recuperação da pesquisa	Aprovado
N04	Busca vazia, limpeza, papel predefinido no cadastro, endereços e larguras de 320, 390, 768 e 1366 pixels	Aprovado

Fonte: elaboração a partir do código e das verificações do projeto (2026).

Além dos testes, o comando de build concluiu a geração da interface, e npm audit não apontou vulnerabilidades conhecidas nos conjuntos de dependências da raiz, API e interface na data da revisão. Esse resultado não corresponde a uma auditoria completa de segurança. A medição local está descrita na seção 2.2.2 e pode variar a cada execução.

A compatibilidade com a base existente foi conferida em uma cópia consistente do arquivo local. Todos os registros anteriores foram comparados campo a campo após a inicialização. A verificação de integridade retornou ok e não foram encontrados erros de chaves estrangeiras. O banco original não foi usado pelos testes.

Quadro 9 – Migração verificada em cópia do banco local

Tabela	Antes	Após inicialização	Registros existentes
users	6	6	Preservados
categories	7	153	Preservados
provider_profiles	6	6	Preservados
provider_categories	12	12	Preservados

Fonte: elaboração a partir do código e das verificações do projeto (2026).

As evidências ficam em docs/evidencias: capturas da interface, benchmark.json e migracao.json. O relatório de validação descreve os comandos. Os resultados se referem aos cenários executados em Windows com navegador Chromium; não houve teste com usuários externos nem teste de carga concorrente em produção.

4.3 Riscos identificados

Quadro 10 – Riscos e limites

Risco ou limite	Impacto	Tratamento atual / evolução
Token no localStorage	Scripts executados na origem podem acessar o token.	React renderiza texto sem HTML bruto; considerar cookies HttpOnly, política de conteúdo e revisão antes de publicar.
Logout sem revogação no servidor	Um token copiado continua válido até expirar.	Expiração de sete dias e verificação de conta ativa; implementar revogação em evolução.
SQLite e pesquisa em memória	Maior carga pode exigir paginação e outro desenho de acesso.	Medição local registrada; paginar e avaliar carga antes de escalar.
Confirmação unilateral de conclusão	O registro do prestador não prova execução presencial.	Limitação explícita; prever confirmação do cliente e tratamento de disputas.

Risco ou limite	Impacto	Tratamento atual / evolução
Ausência de email e moderação	Não há recuperação de senha ou verificação documental.	Não anunciar esses recursos; planejar serviço de email e governança.
Sem pagamento integrado	O orçamento não realiza cobrança.	Pagamento é negociado fora da aplicação; integração futura.
Configuração e publicação	Origem ou segredo incorreto impede acesso ou reduz proteção.	Exemplos de ambiente, falha de configuração em produção e orientação de HTTPS.
Dependências e dados locais	Atualizações podem introduzir mudanças; arquivo local exige cópia de segurança.	Lockfiles, testes após atualização e planejamento de backup.

Fonte: elaboração a partir do código e das verificações do projeto (2026).

4.4 Discussão dos resultados

A evolução principal foi transformar uma vitrine com autenticação em um percurso de atendimento demonstrável. O cliente pesquisa, descreve a necessidade e acompanha a resposta. O prestador recebe o pedido e registra um valor. O aceite libera o contato profissional para o participante, e a avaliação depende de um serviço concluído. Dessa forma, as notas deixam de depender de valores previamente preenchidos no perfil.

O desenho mantém funções e arquivos diretamente associados às responsabilidades do sistema. Não foram introduzidos microsserviços, filas ou novos frameworks de interface. Essa escolha favorece a explicação acadêmica e não implica que a estrutura seja a melhor solução para qualquer volume de uso. A busca em memória e a atualização manual das listas são compromissos conscientes da versão.

Os testes demonstraram o comportamento dos cenários selecionados e verificaram bloqueios que a interface sozinha não poderia garantir. A ausência de falhas nesses testes não prova ausência total de defeitos. A validação futura com pessoas reais, outros navegadores e condições de rede variadas poderá revelar necessidades que não aparecem na execução automatizada local.

5 PERSPECTIVAS FUTURAS

As próximas melhorias devem priorizar uso real e segurança antes de aumentar a complexidade: recuperação de senha com email verificado, revisão do armazenamento e da revogação de sessão, consentimento e gestão de dados pessoais, backup e monitoramento. A publicação exigirá HTTPS e revisão específica das configurações do ambiente.

No fluxo de atendimento, há espaço para confirmação de conclusão pelo cliente, agenda, cancelamento após aceite com regras de acordo, mensagens entre participantes, upload de fotografias e moderação. Pagamentos demandam provedor externo, regras de cobrança e tratamento de estorno. Tais recursos não estão disponíveis na versão descrita.

Caso a base cresça, recomenda-se medir consultas e volume antes de modificar a arquitetura. Paginação, filtragem no banco e índices orientados pelas consultas podem ser etapas anteriores à migração para um banco cliente-servidor. Nenhuma decisão de expansão depende de adotar tecnologias complexas apenas por tendência.

6 CONSIDERAÇÕES FINAIS

O To Work implementa o ciclo acadêmico de conexão entre cliente e prestador com uma interface renovada e as tecnologias da base. Busca, catálogo de 152 categorias, perfil editável, solicitações, orçamento, aceite, conclusão e avaliação compõem um fluxo integrado, sustentado por regras explícitas de autorização e situação. A organização em 12 áreas e a pesquisa por profissão ampliam as possibilidades de atuação sem exigir novas tabelas de cadastro.

O trabalho também produziu documentação, diagramas, capturas e orientações de execução e apresentação. O resultado permite demonstrar conceitos de interface, API, banco relacional, autenticação e testes com uma estrutura organizada para compreensão e manutenção. As verificações realizadas aprovam os cenários documentados e delimitam as funcionalidades que permanecem futuras.

A contribuição se concentra em tornar a proposta executável e explicável, sem apresentar a versão acadêmica como um serviço já preparado para operação pública em larga escala. A validação com usuários e a preparação de publicação constituem os próximos passos naturais do projeto.

REFERÊNCIAS BIBLIOGRÁFICAS

AUTH0. node-jsonwebtoken: documentação do projeto. Documentação eletrônica. Disponível em: https://github.com/auth0/node-jsonwebtoken. Acesso em: 30 out. 2026.

BCRYPT.JS. bcrypt.js: documentação do projeto. Documentação eletrônica. Disponível em: https://github.com/dcodeIO/bcrypt.js. Acesso em: 30 out. 2026.

EXPRESS. Production best practices: security. Documentação eletrônica. Disponível em: https://expressjs.com/en/advanced/best-practice-security/. Acesso em: 30 out. 2026.

GOOGLE. Google Engineering Practices: what to look for in a code review. Documentação eletrônica. Disponível em: https://google.github.io/eng-practices/review/reviewer/looking-for.html. Acesso em: 30 out. 2026.

REACT. Quick Start. Documentação eletrônica. Disponível em: https://react.dev/learn. Acesso em: 30 out. 2026.

SQLITE. Appropriate uses for SQLite [2026a]. Documentação eletrônica. Disponível em: https://www.sqlite.org/whentouse.html. Acesso em: 30 out. 2026.

SQLITE. Transaction [2026b]. Documentação eletrônica. Disponível em: https://www.sqlite.org/lang_transaction.html. Acesso em: 30 out. 2026.

VITE. Getting started. Documentação eletrônica. Disponível em: https://vite.dev/guide/. Acesso em: 30 out. 2026.

UNIFASIPE. Modelo de documentação de Projeto Interdisciplinar. Sinop-MT, [s.d.].

TO WORK. Código-fonte, testes e evidências da revisão de 30 de outubro de 2026. Material do projeto, 2026.

APÊNDICE A – EXECUÇÃO E DEMONSTRAÇÃO

A.1 Preparação local

A execução foi verificada no Windows com Node.js 24.16.0 e npm. Na pasta que contém package.json, backend e frontend, os comandos abaixo instalam as dependências e iniciam a API e a interface. Uma conexão com a internet é necessária para baixar os pacotes; o fluxo principal da aplicação não depende de serviço externo após a instalação.

```
npm install
npm run setup
npm run dev
```

A interface abre em http://localhost:5173 e a API responde em http://localhost:3001/api/health. O comando Ctrl+C encerra os processos iniciados no terminal. As portas devem estar livres. Em um computador com a cópia completa da pasta de trabalho, o arquivo INICIAR_TO_WORK.bat também executa o inicializador do projeto.

As variáveis de ambiente são documentadas nos arquivos .env.example. DATABASE_FILE define o arquivo SQLite; FRONTEND_URL define a origem permitida; PORT define a porta da API e VITE_API_URL define o endereço usado pela interface. JWT_SECRET deve ser uma chave pessoal e não deve ser publicado. Sem essa variável, o desenvolvimento utiliza uma chave temporária; reiniciar a API encerra a validade das sessões emitidas com essa chave. Em produção, o código exige ao menos 32 caracteres no segredo.

A.2 Roteiro de demonstração

1. Cadastre um prestador e preencha cidade, UF, descrição e telefone no perfil profissional.

2. Em Com o que você trabalha?, digite jardineiro, escolha Jardinagem e clique em Vincular categoria. Repita com Elétrica para demonstrar várias áreas no mesmo perfil.

3. Abra uma janela anônima e cadastre um cliente. Use sessões separadas, pois abas do mesmo navegador compartilham o login.

4. No cliente, pesquise o serviço, selecione a cidade ou a categoria e abra o perfil do prestador.

5. Envie uma solicitação com categoria, título, descrição da necessidade e cidade.

6. No prestador, atualize a lista de solicitações e envie um orçamento com o valor combinado.

7. No cliente, atualize a lista e aceite o orçamento. O telefone do prestador passa a ficar disponível nesse pedido.

8. No prestador, atualize a lista e registre a conclusão. No cliente, atualize e envie uma avaliação.

9. Consulte o perfil público para mostrar a nota recebida e recarregue a página para demonstrar a persistência.

As solicitações são atualizadas pelo botão Atualizar solicitações. A versão não possui atualização em tempo real. No painel, ativar e desativar uma categoria controla sua disponibilidade pública sem apagar o vínculo nem os pedidos anteriores.

A.3 Verificação e estudo

Os comandos npm test, npm run build e npm run test:e2e repetem as verificações principais. Para o navegador, instale antes o Chromium com npx playwright install chromium. Os testes usam bancos temporários e as portas 3107 e 5187, preservando o arquivo de dados da utilização normal.

Para explicar a implementação, estude App.jsx e AuthContext.jsx na interface; authRoutes.js e authMiddleware.js para autenticação; requestRoutes.js para as decisões sobre pedidos; database.js e schema.sql para persistência; serviceCategories.json e CategoryOptions.jsx para o catálogo. O guia GUIA_APRESENTACAO.md complementa o roteiro com perguntas e respostas em linguagem simples.

APÊNDICE B – CATÁLOGO PADRÃO DE SERVIÇOS

A relação abaixo foi gerada diretamente do arquivo serviceCategories.json da versão documentada. São 152 categorias em 12 grupos. Os grupos organizam a navegação; a disponibilidade de cada serviço depende dos vínculos ativos dos prestadores. Categorias personalizadas herdadas de versões anteriores não fazem parte desta relação padrão.

Quadro 11 – Relação completa do catálogo padrão

Área de atuação	Categorias incluídas
Construção e reformas	Elétrica; Hidráulica; Pintura; Pedreiro; Servente de obras; Azulejista; Gesseiro; Drywall; Carpinteiro; Marceneiro; Serralheiro; Soldador; Vidraceiro; Telhadista; Calheiro; Impermeabilização; Marmoraria; Instalação de pisos; Papel de parede; Arquitetura; Engenharia civil; Design de interiores; Topografia; Terraplanagem.
Limpeza e organização	Limpeza; Limpeza pós-obra; Limpeza de estofados; Limpeza de vidros; Limpeza de caixas d'água; Limpeza de piscinas; Lavagem de tapetes e cortinas; Lavanderia e passadoria; Organização de ambientes; Controle de pragas; Desentupimento; Coleta de entulho.
Jardim, campo e áreas externas	Jardinagem; Paisagismo; Poda de árvores; Roçagem de terrenos; Irrigação; Horta e cultivo; Serviços rurais; Cercas e alambrados; Manutenção de áreas de lazer.
Instalação e manutenção	Marido de aluguel; Montagem de móveis; Chaveiro; Ar-condicionado; Refrigeração; Conserto de eletrodomésticos; Conserto de eletrônicos; Antenas e televisão; Câmeras e alarmes; Portões automáticos; Interfones; Energia solar; Automação residencial; Instalação de cortinas e persianas; Redes e telas de proteção.
Tecnologia e informática	Informática; Manutenção de computadores; Manutenção de notebooks; Conserto de celulares; Redes e Wi-Fi; Impressoras; Desenvolvimento de sites; Desenvolvimento de sistemas; Suporte técnico remoto; Planilhas e automação; Recuperação de dados; Treinamento em informática.
Transporte e serviços automotivos	Fretes; Mudanças; Motoboy e entregas; Motorista particular; Mecânica automotiva; Autoelétrica; Mecânica de motos; Borracharia; Lavagem de veículos; Estética automotiva; Funilaria e pintura automotiva; Manutenção de bicicletas.
Beleza e bem-estar	Cabeleireiro; Barbeiro; Manicure e pedicure; Maquiagem; Design de sobrancelhas; Extensão de cílios; Depilação; Estética facial e corporal; Massoterapia; Personal trainer; Yoga e pilates.
Eventos, alimentação e turismo	Cozinheiro; Marmitas e refeições; Confeitaria; Salgados e lanches; Churrasqueiro; Buffet e catering; Garçom; Bartender; Organização de eventos; Decoração de festas; Fotografia; Filmagem; DJ e sonorização; Músicos e bandas; Animação de festas; Cerimonial; Guia de turismo.

Área de atuação	Categorias incluídas
Aulas e educação	Reforço escolar; Matemática; Português e redação; Idiomas; Música; Dança; Esportes; Preparação para provas; Artes e artesanato; Aulas de culinária.
Cuidados, família e animais	Babá; Cuidador de idosos; Acompanhante de pessoas; Cuidador de animais; Passeador de cães; Banho e tosa; Adestramento; Transporte de animais; Hospedagem de animais.
Moda, costura e artesanato	Costura e ajustes; Alfaiataria; Bordado; Crochê e tricô; Sapataria; Tapeçaria; Artesanato personalizado; Personalização de produtos.
Comunicação, design e negócios	Design gráfico; Edição de vídeo; Edição de fotos; Social media; Marketing digital; Redação e conteúdo; Revisão de textos; Tradução e interpretação; Assistente virtual; Digitação e cadastro; Serviços administrativos; Contabilidade; Consultoria empresarial.

Fonte: elaboração a partir do código e das verificações do projeto (2026).